Transformer Architectures

Overview

- Architecture based on:
  - Attention
  - Feedforward networks
- Processes sequences in parallel (unlike RNNs)
- Used in NLP, computer vision, and more

Architecture

Encoder

- Input → contextual representations
- Each token attends to all tokens (bidirectional)

Decoder

- Generates output sequence
- Attends to:
  - Previous outputs
  - Encoder outputs

Positional Encoding

- Adds order information to tokens
- Required because transformers do not process sequentially

Multi-Head Attention

- Multiple attention heads run in parallel
- Each head has its own Q, K, V

- Each head captures different relationships
- Result: richer representations

Pretrained Models

BERT (Bidirectional Encoder Representations from Transformers)

- Encoder-only architecture
- Bidirectional attention (left + right context)
- Used for:
  - Text classification
  - Sentiment analysis
  - Question answering
  - Named entity recognition
- Pretrained on Wikipedia and BookCorpus

GPT (Generative Pretrained Transformer)

- Decoder-only architecture
- Causal attention (left context only)
- Autoregressive (predicts next token)
- Used for text generation and chat systems

Input Preprocessing

Tokenization (BPE)

- Byte Pair Encoding (BPE)
- Frequent words remain whole
- Rare words split into subwords

Example:
strawberrymilkshake → ["straw", "berry", "milk", "shake"]

Context Window

- Maximum number of tokens:
  - BERT: 512
  - GPT-2: 1024
  - GPT-4o: 128000

Truncation and Padding

- Long sequences → truncated
- Short sequences → padded with [PAD]
- Enables batching

Special Tokens

[CLS]

- Added at beginning
- Represents entire sequence
- Used for classification

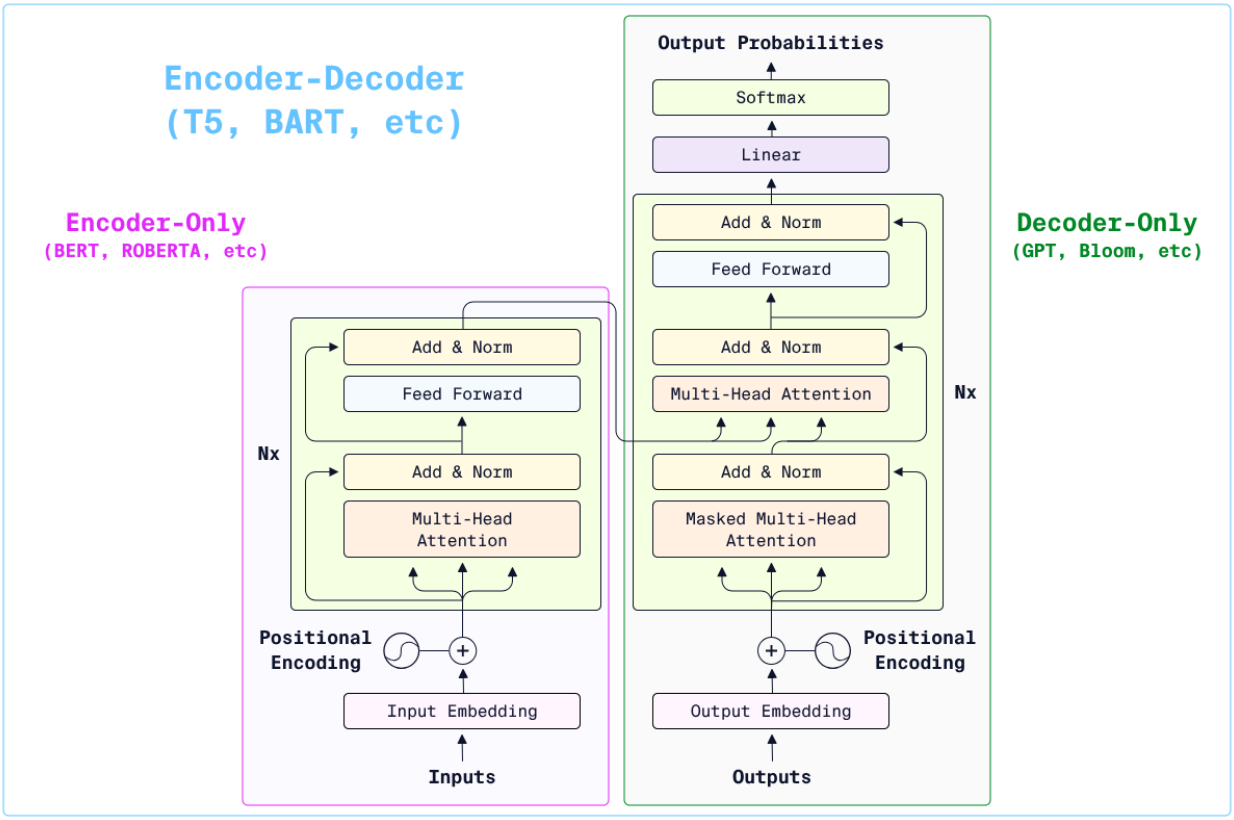
[SEP]

- Separates sequences
- Marks end of input

Examples

Single sequence:
["[CLS]", "The", "black", "cat", "eats", "pumpkin", "[SEP]"]

Two sequences:
["[CLS]", "The", "black", "cat", "eats", "pumpkin",
"[SEP]", "The", "fluffy", "dog", "wags", "happily", "[SEP]"]



Key Points

- Attention replaces recurrence
- Encoder = understanding
- Decoder = generation
- Multi-head attention captures multiple relationships
- Requires positional encoding and structured inputs